

Charitan - A Global Charity Donation Platform - Milestone 2

EEET2582 | Software Engineering: Architecture and Design

Tiger Squad - Team B

Number of pages: 18

Students:

Tran Xuan Hoang Dat - s3651550,

Nguyen Duc Anh - s3878010,

Dang Vinh Luan - s3927222,

Le Thanh Vinh - s3914997,

Luong Gia Bao - s3915203

1. Introduction.....	2
2. Key Adjustments.....	2
3. Services Breakdown.....	4
1. Zookeeper Service.....	4
Image:.....	4
Purpose:.....	4
Key Features:.....	4
2. Kafka Service.....	4
Image:.....	4
Purpose:.....	5
Key Features:.....	5
3. Donor-Service.....	5
Build Context:.....	5
Purpose:.....	5
Dependencies:.....	5
Key Features:.....	6
4. Charity-Service.....	6
Build Context:.....	6
Purpose:.....	6
Dependencies:.....	6
Key Features:.....	6
5. Authentication-Service.....	7
Build Context:.....	7
Purpose:.....	7
Dependencies:.....	7
Key Features:.....	7
4. C4 Diagram.....	8
Container Diagram.....	8
Workflow.....	9
Donor-Service.....	10
Key Components and Responsibilities.....	10
Workflow Explanation.....	11
Charity-Service.....	12
Key Components and Responsibilities.....	12
Diagram with JWT Token Integration.....	13
Legend.....	13
Key Containers and Responsibilities.....	13
Statistic Service.....	15
Key Components and Responsibilities.....	15
Authentication Component.....	17
Legend.....	17
Key Components and Responsibilities.....	17
5. Conclusion.....	18

1. Introduction

Following the project's initial request and a detailed consultation with our professor, Team Tiger B has undertaken significant adjustments to align with the provided feedback and objectives. These changes were implemented to ensure that the project not only meets academic standards but also demonstrates practical relevance, enhancing its overall functionality and application in real-world scenarios.

2. Key Adjustments

Realignment with Feedback

Refining Scope and Structure:

- Incorporated detailed suggestions from our professor to ensure the project meets academic expectations and practical use cases.
- Streamlined objectives to focus on implementing a scalable and modular microservice architecture.

Architectural Adjustments:

- Redesigned the system to prioritize modularity, ensuring each service functions independently while contributing to the overall ecosystem.
- Improved scalability by adopting distributed service patterns and reducing interdependencies between components.

Backend Overhaul

Rebuilding the Backend:

- Reconstructed the backend to transition from a monolithic to a microservices-based architecture.

- Segregated large, complex components into manageable and independent services to improve efficiency.

Inter-Service Communication:

- Introduced **Kafka** for event-driven architecture, allowing asynchronous communication and better reliability between microservices.
- Ensured smooth data flow between services by defining clear communication protocols and integration points.

Adoption of New Frameworks and Tools

Modern Frameworks:

- Integrated industry-standard frameworks such as **Express.js** (Node.js-based systems) for faster development and better scalability.
- Leveraged these frameworks' features, such as dependency injection and built-in middleware, to reduce complexity.

Containerization:

- Used **Docker Compose** for containerization, ensuring simplified service orchestration and deployment.
- Established consistent environments for local development and production.

Authentication and Security:

- Enhanced authentication mechanisms by implementing secure key management using technologies like **JWT**, **RSA keys**, and **encryption services**.
- Centralized security features to streamline user verification and token management.

3. Services Breakdown

1. Zookeeper Service

Image:

- `confluentinc/cp-zookeeper:latest`

Purpose:

- Provides centralized **coordination** for distributed systems, primarily Kafka.
- Handles **configuration management**, **synchronization**, and **leader election**.

Key Features:

- **Environment Variables:**
 - `ZOOKEEPER_CLIENT_PORT`: Port for client connections.
 - `ZOOKEEPER_TICK_TIME`: Configures heartbeat intervals and timeout detection.
- **Dynamic Port Mapping:**
 - `${ZOOKEEPER_PORT}` supports flexible port allocation.
- **Ports Exposed:**
 - `${ZOOKEEPER_PORT}` for client communication.
- **Standalone Service:**
 - No direct dependencies; other services (e.g., Kafka) depend on Zookeeper.

2. Kafka Service

Image:

- `confluentinc/cp-kafka:latest`

Purpose:

- Functions as a **distributed event streaming platform**.
- Facilitates **message brokering** between services.

Key Features:

- **Listener Types:**
 - **INTERNAL:** For inter-service communication.
 - **EXTERNAL:** For external access via `${KAFKA_EXTERNAL_ADVERTISED_HOSTNAME}`.
- **Configurable Settings:**
 - `KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR`: Ensures replication for fault tolerance.
 - Supports plaintext communication via protocol mapping.
- **Volumes:**
 - Maps `./kafka/config/server.properties` to `/etc/kafka/server.properties`.
- **Ports Exposed:**
 - `${KAFKA_PORT}:${KAFKA_PORT}` (internal).
 - `9093:9093` (external).

3. Donor-Service

Build Context:

- `./services/donor-service`

Purpose:

- Manages **donor-related operations** and data.

Dependencies:

- Relies on Kafka for **message-driven communication**.

Key Features:

- **Environment Variables:**
 - **KAFKA_BROKER**: Kafka broker endpoint.
 - **MONGO_URI**: MongoDB connection URI.
- **Volumes:**
 - Mounts:
 - **./services/donor-service** (service code).
 - Local **node_modules** for dependency management.
- **Command:**
 - Executes service with **npm start**.

4. Charity-Service

Build Context:

- **./services/charity-service**

Purpose:

- Manages **charity operations** and integrates donor data.

Dependencies:

- Kafka for **event-driven communication**.

Key Features:

- **Environment Variables:**
 - **KAFKA_BROKER**: Kafka broker endpoint.
 - **MONGO_URI**: MongoDB connection URI.
- **Volumes:**
 - Mounts:
 - **./services/charity-service** (service code).

- Local `node_modules` for runtime updates.
- **Command:**
 - Runs service with `npm start`.

5. Authentication-Service

Build Context:

- `./services/authentication-service`

Purpose:

- Manages **user authentication** using token-based security.

Dependencies:

- Kafka, Charity-Service, and Donor-Service for messaging and user operations.

Key Features:

- **Environment Variables:**
 - `KAFKA_BROKER`: Kafka broker endpoint.
 - `MONGO_URI`: MongoDB connection URI.
 - **Cryptographic Keys:**
 - `JWS_PRIVATE_KEY_PATH`, `JWE_PUBLIC_KEY_PATH`,
`RSA_PRIVATE_KEY_PATH`.
- **Volumes:**
 - Mounts:
 - `keys` directory for cryptographic operations.
 - Service code and `node_modules`.

4. C4 Diagram

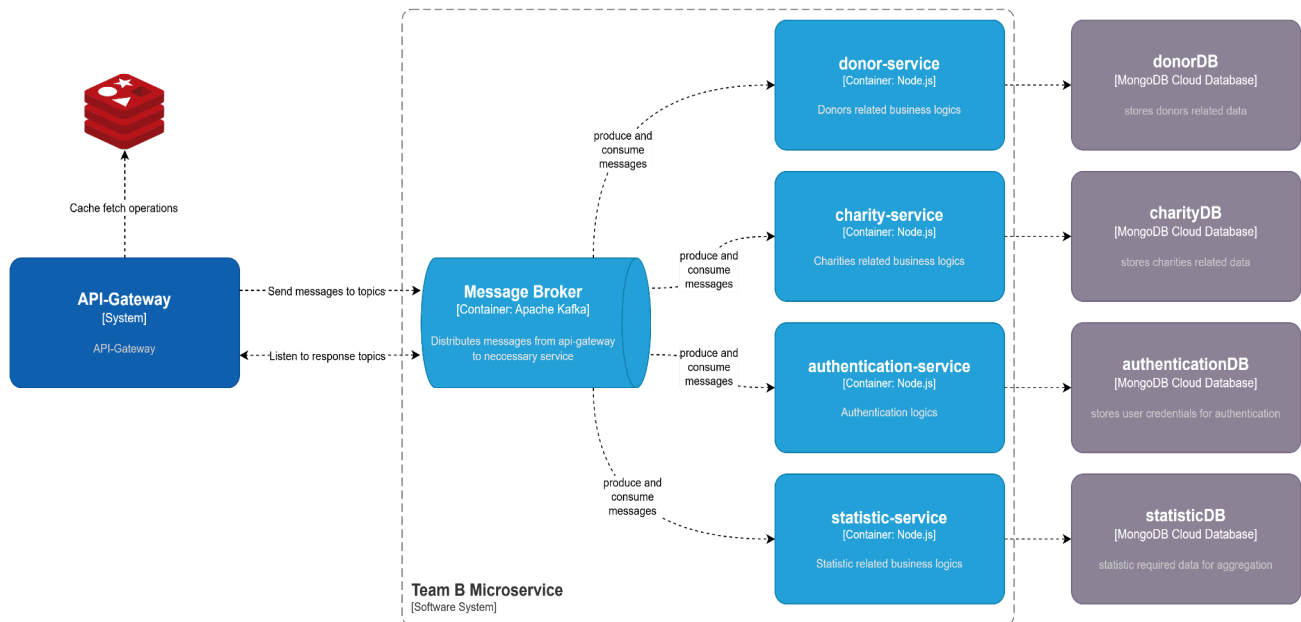


Fig 1: [Container Diagram](#)

Container Diagram

1. API Gateway (System)

- Acts as the single entry point for all client requests.
- Responsibilities:
 - Routes incoming requests to the appropriate microservices.
 - Performs caching operations with **Redis** to improve response times.
 - Communicates with the **Message Broker** to forward requests and handle responses.
- Interactions:
 - Directly interacts with **Redis** for cache operations.
 - Publishes and listens to topics on **Message Broker (Apache Kafka)**.

2. Message Broker (Apache Kafka)

- Handles communication between the API Gateway and microservices.
- Responsibilities:
 - Distributes messages from the API Gateway to the necessary services.
 - Implements a publish-subscribe model to produce and consume messages between components.

3. Microservices

- Each microservice is implemented in **Node.js** and deployed in a separate container.
- Key microservices:
 - **donor-service**: Manages donor-related business logic and interacts with **donorDB**.
 - **charity-service**: Handles charity-related operations and connects to **charityDB**.
 - **authentication-service**: Manages user authentication processes and interacts with **authenticationDB**.
 - **statistic-service**: Handles aggregation and statistics-related tasks, relying on **statisticDB**.

4. Databases

- Each database is hosted in the **MongoDB Cloud**, providing storage for individual services.
- Databases:
 - **donorDB**: Stores data related to donors.
 - **charityDB**: Stores information on charities.
 - **authenticationDB**: Contains user credentials and authentication details.
 - **statisticDB**: Stores aggregated data for statistics and reporting.

5. Redis Cache

- A caching layer to store temporary data for quick retrieval, primarily used by the API Gateway.

Workflow

1. A client sends a request to the **API Gateway**.
2. The **API Gateway** checks **Redis** for cached data.
 - If cached data is available, it responds immediately.
 - Otherwise, it forwards the request to the **Message Broker**.
3. The **Message Broker** routes the request to the appropriate microservice based on the topic.
4. The microservice processes the request, interacting with its corresponding database if necessary.
5. Processed data is returned via the **Message Broker** to the **API Gateway**, which sends the response back to the client.

This architecture ensures:

- **Scalability**: Each service is independent and can scale individually.
- **Loose Coupling**: Services communicate through Kafka topics, minimizing dependencies.
- **Improved Performance**: Caching with Redis reduces latency for frequent operations.

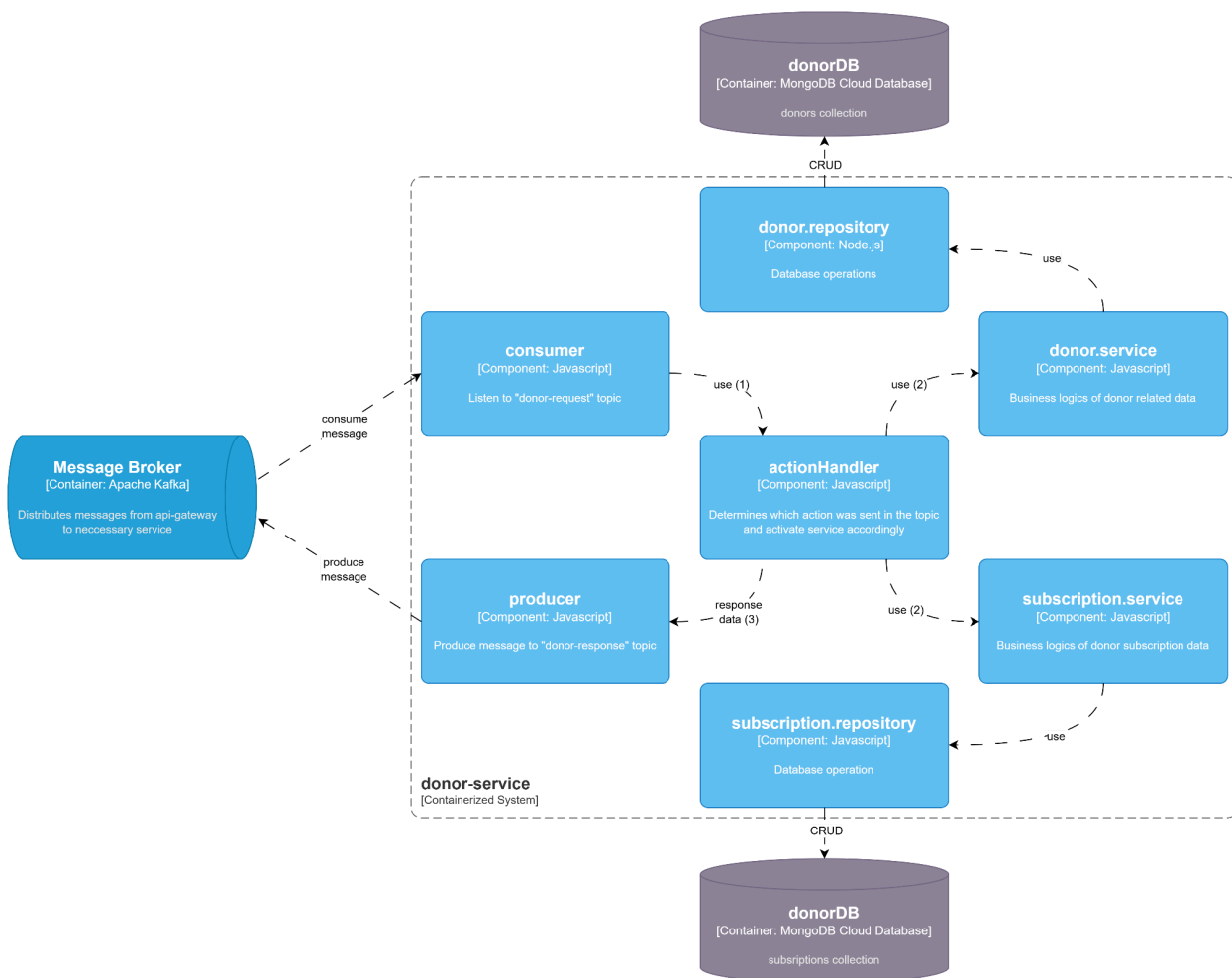


Fig 2: Donor Component Diagram

Donor-Service

This diagram focuses on the **donor-service** container, illustrating the internal components and how they interact with each other, the **Message Broker**, and the **donorDB**.

Key Components and Responsibilities

1. Message Broker (Apache Kafka)

- Facilitates communication by:

- **Producing Messages:** Sent by the **producer** to the "donor-response" topic.
- **Consuming Messages:** Listened by the **consumer** on the "donor-request" topic.

2. Components within donor-service

- **consumer:**

- A JavaScript component that listens to the **"donor-request"** topic for incoming messages.
 - Passes the received messages to the **actionHandler** for processing.
 - **producer:**
 - A JavaScript component that produces messages to the **"donor-response"** topic, responding with results.
 - **actionHandler:**
 - A central component that determines the action required based on the message received from the **consumer**.
 - Delegates tasks to relevant services:
 - **donor.service**
 - **subscription.service**
 - **donor.service:**
 - Handles core business logic related to donor data, such as creating, updating, or retrieving donor information.
 - Uses the **donor.repository** for database operations.
 - **subscription.service:**
 - Manages donor subscriptions, handling related business logic.
 - Relies on the **subscription.repository** for database interactions.
 - **donor.repository:**
 - A Node.js component responsible for performing CRUD operations on the **donorDB** (specifically, the **donors collection**).
 - **subscription.repository:**
 - Responsible for CRUD operations on the **subscriptions collection** in the **donorDB**.
3. **donorDB (MongoDB Cloud Database)**
- The central storage for donor-related data, consisting of:
 - **donors collection:** Stores donor details.
 - **subscriptions collection:** Stores subscription details for donors.

Workflow Explanation

1. The **consumer** listens to messages from the "donor-request" topic via the **Message Broker**.
2. Upon receiving a message, the **actionHandler** determines the required action:
 - If donor data is needed, it uses **donor.service**.
 - If subscription data is required, it uses **subscription.service**.
3. Each service interacts with its respective repository (**donor.repository** or **subscription.repository**) to fetch or modify data in the **donorDB**.
4. The results are passed back to the **producer**, which sends a response message to the "donor-response" topic in the **Message Broker**.

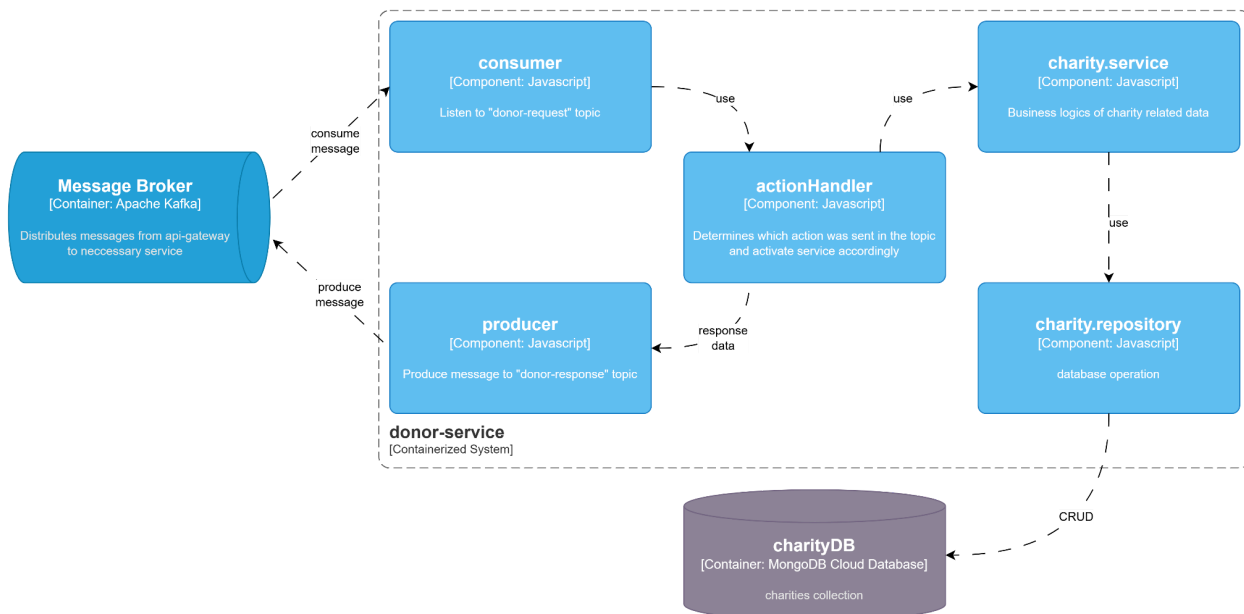


Diagram 3: [Charity Component Diagram](#)

Charity-Service

Key Components and Responsibilities

1. Message Broker (Apache Kafka)

- Facilitates asynchronous communication by:
 - **Consuming Messages:** The **consumer** listens to the "donor-request" topic for incoming messages.
 - **Producing Messages:** The **producer** publishes results to the "donor-response" topic.

2. Components within charity-service

- **consumer:**
 - A JavaScript component that listens to the "**donor-request**" topic for incoming messages.
 - Passes the received messages to the **actionHandler** for processing.
- **producer:**
 - A JavaScript component that produces messages to the "**donor-response**" topic, sending processed data back.
- **actionHandler:**
 - Determines the appropriate service (e.g., **charity.service**) to handle the received message.
 - Delegates the request to the correct service based on the action.
- **charity.service:**

- Contains business logic for managing charity-related data, such as retrieving, updating, or deleting charity records.
 - Utilizes the **charity.repository** for database operations.
 - **charity.repository:**
 - Responsible for CRUD operations on the **charityDB** (specifically, the **charities collection**).
3. **charityDB (MongoDB Cloud Database)**
- A database that stores all charity-related data in a dedicated **charities collection**.

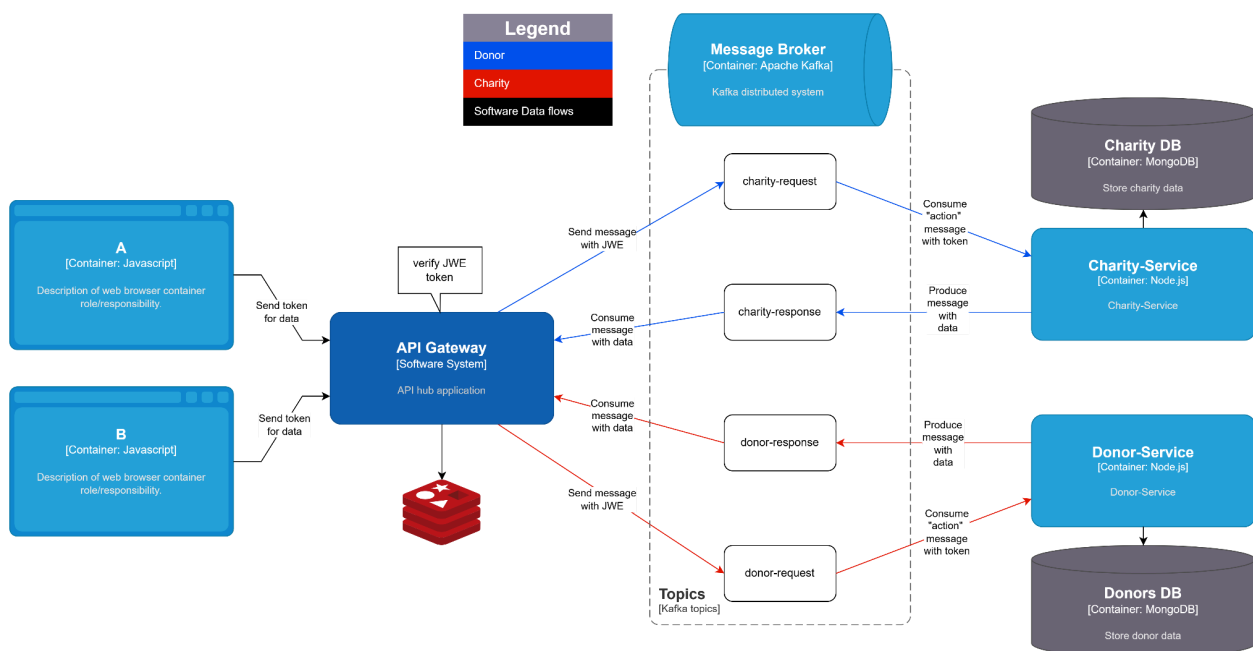


Fig 4: Diagram with JWT Token Integration

Diagram with JWT Token Integration

Legend

- **Blue Flows:** Represent **Charity operations**.
- **Red Flows:** Represent **Donor operations**.
- **JWE Token:** JSON Web Encryption token used for secure communication.

Key Containers and Responsibilities

1. Front-End Applications (Containers A and B)

- A: Represents the Donor and Charity front-end application.
 - Responsibilities:

- Sends JWE tokens to authenticate and request data from the **API Gateway**.
- **B**: Represents the Admin front-end application.
 - Similar responsibilities for admin-specific operations.
- 2. **API Gateway (Software System)**
 - Acts as the central hub for all requests and responses:
 - **Verification**: Verifies incoming JWE tokens for authentication and authorization.
 - **Routing**: Sends authenticated requests to **Kafka topics** based on the operation type (e.g., "charity-request" or "donor-request").
 - **Message Handling**: Consumes messages from Kafka topics (e.g., "charity-response", "donor-response") to send data back to the clients.
- 3. **Message Broker (Apache Kafka)**
 - Facilitates asynchronous communication between the API Gateway and microservices:
 - **charity-request/donor-request**: Topics for sending operation-specific messages.
 - **charity-response/donor-response**: Topics for receiving processed data.
- 4. **Microservices**
 - **Charity-Service (Node.js)**:
 - Handles charity-related actions like creating, updating, and retrieving charity data.
 - Interacts with **Charity DB (MongoDB)** to perform CRUD operations.
 - Produces processed results back to the "charity-response" topic.
 - **Donor-Service (Node.js)**:
 - Manages donor-related actions, including registration and data retrieval.
 - Uses **Donors DB (MongoDB)** for CRUD operations.
 - Sends results to the "donor-response" topic.
- 5. **Databases**
 - **Charity DB (MongoDB)**: Stores charity-related data (e.g., charity profiles).
 - **Donors DB (MongoDB)**: Stores donor-related data (e.g., donor profiles and donations).
 - Data access is secured and abstracted via the respective services.
- 6. **Redis (Caching)**
 - Used by the **API Gateway** for temporary caching of frequent data, improving response times.

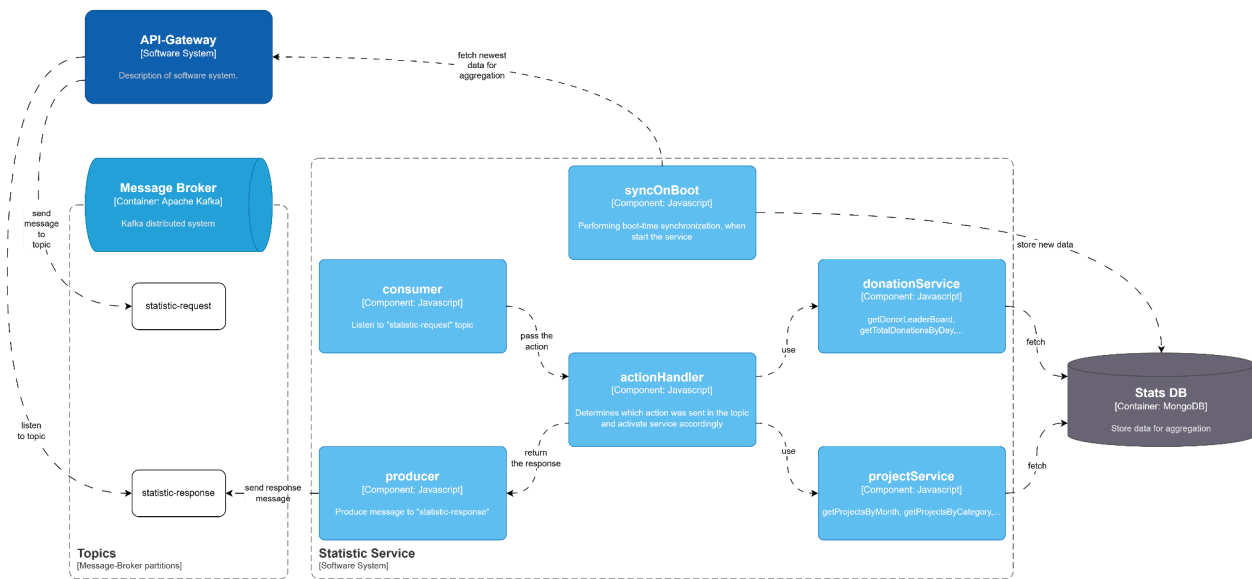


Fig 5: Statistic Service

Statistic Service

Key Components and Responsibilities

1. **API Gateway (Software System)**
 - Routes user requests to the appropriate Kafka topic (e.g., "statistic-request") for processing by the Statistic Service.
2. **Message Broker (Apache Kafka)**
 - Acts as a distributed messaging system:
 - **statistic-request**: Topic where the API Gateway publishes requests for statistical data.
 - **statistic-response**: Topic where the Statistic Service publishes the results after processing.
3. **Statistic Service**
 - **consumer**:
 - Listens to the "statistic-request" topic for incoming statistical data aggregation requests.
 - Passes the request to the **actionHandler** for processing.
 - **actionHandler**:
 - Decides the type of action based on the request (e.g., fetching donation data, retrieving project statistics).
 - Delegates tasks to the appropriate service (**donationService** or **projectService**).
 - **producer**:

- Publishes the processed results to the "statistic-response" topic, making the data available to the API Gateway.

4. Supporting Services

- **syncOnBoot:**
 - Performs synchronization tasks when the Statistic Service starts, ensuring it has the latest data for accurate aggregation.
- **donationService:**
 - Handles operations related to donor statistics, such as:
 - `getDonorLeaderboard`: Fetches the top donors.
 - `getTotalDonationsByDay`: Aggregates donations by day.
 - Interacts with **Stats DB** to fetch and store donation-related data.
- **projectService:**
 - Focuses on project-related statistics, such as:
 - `getProjectsByMonth`: Retrieves projects grouped by month.
 - `getProjectsByCategory`: Fetches project data based on categories.
 - Communicates with **Stats DB** for aggregation operations.

5. Stats DB (MongoDB)

- Stores aggregated data for efficient retrieval during statistical operations.
- Acts as the primary data source for both **donationService** and **projectService**.

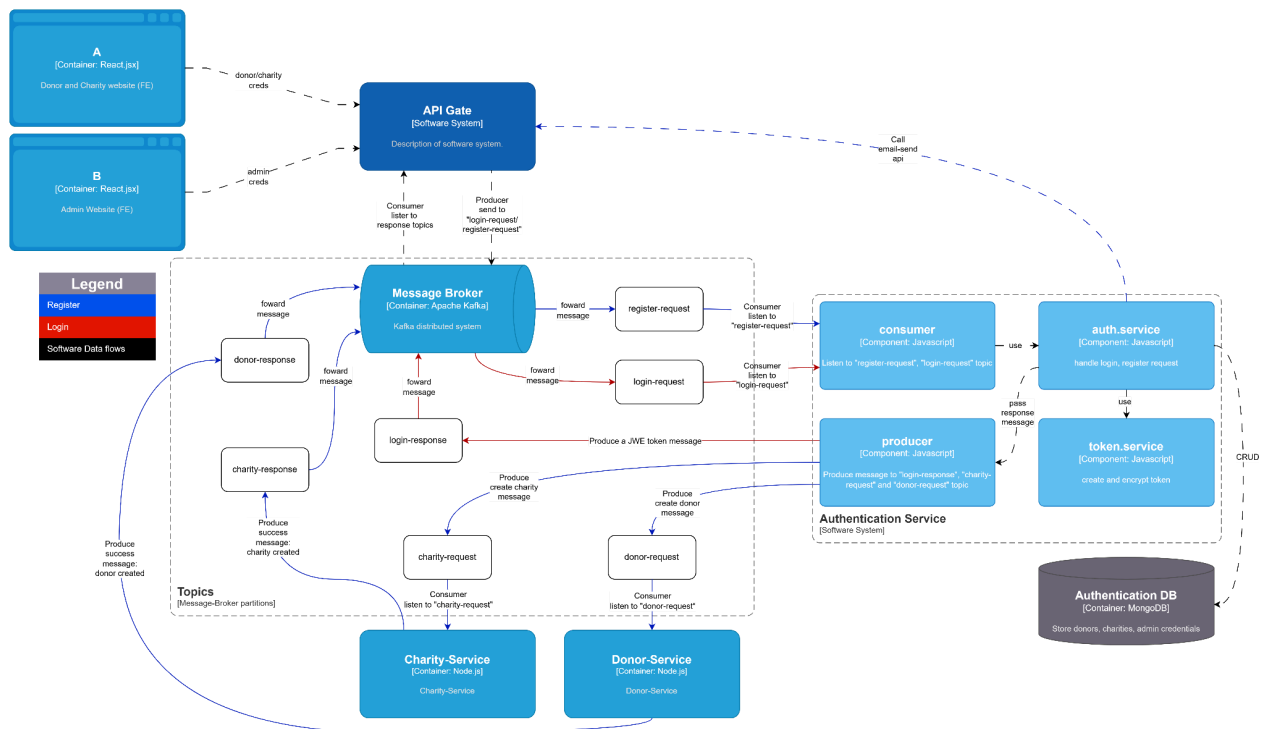


Fig 6: Authentication Component

Authentication Component

Legend

- **Blue Arrows:** Represent the data flow for **Register** operations.
- **Red Arrows:** Represent the data flow for **Login** operations.
- **Topics:** Represent partitions within the **Message Broker** for different operations.

Key Components and Responsibilities

1. Front-End Applications

- **A (Donor and Charity Website):**
 - Built using React.js.
 - Interacts with the **API Gate** to handle donor and charity-related operations (e.g., registration, donation).
- **B (Admin Website):**
 - Also built using React.js.
 - Used for administrative operations, likely requiring credentials validated through the **Authentication Service**.

2. API Gateway (Software System)

- Acts as the central entry point for all front-end applications.
- Routes requests to appropriate topics in the **Message Broker** (e.g., "login-request", "register-request").

3. Message Broker (Apache Kafka)

- Centralized message distribution system for asynchronous communication.
- Topics include:
 - **register-request:** Handles user registration messages.
 - **login-request:** Handles login messages.
 - **charity-request:** Processes charity-related actions.
 - **donor-request:** Handles donor-related actions.

4. Authentication Service

- Consists of:
 - **consumer:** Listens to "register-request" and "login-request" topics.
 - **auth.service:** Handles the core logic for registration and login operations.
 - **token.service:** Creates and encrypts tokens (e.g., JSON Web Tokens) for user authentication.
 - **producer:** Produces response messages to relevant topics (e.g., "login-response").
- Interacts with **Authentication DB** to store or validate credentials.

5. Charity Service

- Processes requests related to charities:
 - Listens to "charity-request" via a Kafka consumer.
 - Produces success messages after completing charity actions.
- Interacts with the **charityDB** for CRUD operations on charity data.

6. Donor Service

- Handles donor-related operations:
 - Listens to "donor-request" via a Kafka consumer.
 - Produces messages on successful donor-related actions.
- Interacts with the **donorDB** for CRUD operations.

7. Authentication DB (MongoDB)

- Stores:
 - Donor credentials.
 - Charity information.
 - Admin credentials.

5. Conclusion

The Charitan Admin Web Application is designed to provide administrators with comprehensive tools to monitor and manage the end-user donation platform. Built on a microservice architecture, the system ensures scalability, maintainability, and security. Key components, such as MongoDB for flexible data storage, Redis for efficient caching, Kafka for reliable and asynchronous message processing in the backend, and Cloudinary for storing and rendering images, enable seamless operation and real-time monitoring. This design creates a secure environment for administrators, ensuring the end-user web application operates smoothly.

Potential Improvements

Scalability: Introduce replicas for **kafka** and **zookeeper** to ensure high availability.

Logging: Add centralized logging support (e.g., Elasticsearch, Kibana) for monitoring service performance.

Load Balancing: Integrate a load balancer (e.g., Nginx) to manage traffic to microservices.